claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_e2a9aaa7-6b2\agent-a292af264ff1fc761.jsonl`
- SHA-256: `7265a9caec91851de82beb0d9bf3ff6810fce635fe6cc15ec72e8f84c2b1af70`
- Source modified: `2026-05-30T19:10:36+00:00`
- Imported at: `2026-07-08T15:59:24+00:00`
- project: `wf_e2a9aaa7-6b2`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-30T19:10:00.179Z)

Decompose this research question into complementary search angles.

Question

Find promising AND reliable open-source tools/libraries for parsing Indian bank statement PDFs ? especially HDFC Bank savings/transaction statements ? into structured transaction rows (date, narration, ref, value date, withdrawal, deposit, closing balance), for reuse in a LOCAL-FIRST Python personal-finance tool ("muneem").

HARD CONSTRAINTS (a tool that violates these is unusable for us):
1. Must run 100% LOCALLY. Bank statements are highly sensitive ? NO cloud APIs, NO paid SaaS (e.g. bankstatementconverter.com, docparser, LLM extraction APIs). Paid/cloud is not acceptable even as a default.
2. Must be usable from Python.
3. License MUST be permissive & commercially safe: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL or that depends on copyleft native components ? in particular Camelot/tabula's reliance on Ghostscript (AGPL) or a Java runtime. PyMuPDF (AGPL) is already banned for us.

Specifically evaluate and compare these table-extraction approaches, for each giving: license (AND license of native deps), maintenance status (latest release + activity as of 2025/2026), reliability on SEMI-RULED / partially-borderless tables (HDFC's transaction table is not fully line-ruled), and password-protected-PDF support:
- pdfplumber (we already use it; MIT)
- Camelot ? camelot-py vs the maintained "pypdf_table_extraction" fork; its Ghostscript/OpenCV deps
- tabula-py (tabula-java + JVM)
- pypdfium2-based table extraction
- any other notable local options

Also assess the user's idea: "convert the PDF table to Excel/CSV first, then parse" ? is an intermediate CSV/Excel extraction more reliable than direct coordinate/text parsing, and which LOCAL+permissive tools do PDF?CSV/Excel table conversion well?

Finally, search for any existing, maintained, permissively-licensed open-source projects specifically for parsing Indian bank statements (HDFC/ICICI/SBI/Axis) or bank statements generally, that we could reuse or learn from (GitHub/PyPI).

Deliver a ranked shortlist of the most promising + reliable + license-safe options for HDFC statement parsing in a local Python tool, each with its license (and native-dep licenses), maintenance status, strengths/weaknesses for this use case, and a concrete recommendation for muneem (reuse library X vs. hand-roll coordinate clustering with pdfplumber). Cite sources.

Task

Generate 5 distinct web search queries that together cover the question from different angles.

Pick angles that suit the question's domain. Examples:
- broad/primary · academic/technical · recent news · contrarian/skeptical · practitioner/implementation
- For medical: anatomy · common causes · serious differentials · authoritative refs · red flags
- For tech: state-of-art · benchmarks · limitations · industry adoption · cost/tradeoffs

Make queries specific enough to surface high-signal results. Avoid redundancy.
Return: the question (verbatim or lightly normalized), a 1-2 sentence decomposition strategy, and the angles.

Structured output only.

2. assistant (2026-05-30T19:10:02.697Z)

I'll decompose this research question into complementary search angles. Let me first quickly check the project's existing context to ground the queries.

3. user (2026-05-30T19:10:03.288Z)

DESIGN.md

4. user (2026-05-30T19:10:03.509Z)

docs\prior-art-email-mining.md

5. assistant (2026-05-30T19:10:10.712Z)

The files exist as referenced. I have enough context from the question itself and the CLAUDE.md to decompose this well. The question is highly specific: local-first, Python, permissive-license PDF table extraction for Indian (HDFC) bank statements. Let me construct 5 complementary search angles.

The key sub-problems to cover:
1. The specific library license/maintenance comparison (pdfplumber, Camelot/pypdf_table_extraction, tabula-py, pypdfium2) ? the core comparison.
2. The Camelot Ghostscript/AGPL native-dependency landmine specifically (this is the load-bearing license risk).
3. Semi-ruled/borderless table extraction reliability ? the technical hard part for HDFC tables.
4. Password-protected PDF support across local Python libraries (Indian bank PDFs are encrypted).
5. Existing open-source Indian bank statement parsers to reuse/learn from.

The user also raised the "PDF?CSV/Excel intermediate then parse" idea, which folds naturally into the table-extraction angle but deserves its own surfacing.

6. user (2026-05-30T19:10:30.382Z)

Structured output provided successfully

7. assistant (2026-05-30T19:10:36.393Z)

I've decomposed the research question into 5 complementary search angles, returned via the StructuredOutput tool.

The angles cover: (1) the core library bake-off, (2) the copyleft native-dependency disqualifier risk (Ghostscript/AGPL, tabula-java/GPL), (3) the semi-ruled/borderless table reliability problem specific to HDFC, (4) password-protected PDF decryption support, and (5) discovery of existing reusable Indian/generic bank-statement parsers. The "convert to CSV/Excel first" sub-question is absorbed into angles 1 and 5.

Relevant project context files confirmed present: `C:\Users\avidu\Projects\muneem\DESIGN.md` and `C:\Users\avidu\Projects\muneem\docs\prior-art-email-mining.md`.